Data Poisoning Attack on Pruned Neural Networks

Janani Venkatramani **Sherine Chally** **Diya Lakhani** **Anvi Penmetsa**

Abstract

Data poisoning attacks typically assume a fixed model architecture, overlooking structural changes introduced by model pruning. In this paper, we aim to design poisons that leave the full-parameter model largely unaffected, however, become highly effective once pruning is applied. We designed a clean-label data poisoning attack that generates a poison for the data points using a modified U-Net network and utilize random unstructured pruning for our victim network. While we observe drops in accuracy for the pruned networks, our results indicate that these effects cannot be conclusively attributed to the poisoning mechanism, suggesting that pruning-sensitive poisoning remains challenging and warrants further investigation. GitHub Source Code: `https://github.com/anvi-pen/CS260D_Final_Project`

1 Introduction

1.1 The Context

Neural Network Pruning has become a standard compression technique. The primary motivation behind this is to remove the the neural network connections that are considered unimportant, thereby reducing the model size and the energy consumption while maintaining the performance. There are two types of pruning:

- **Unstructured Pruning**: Removes the individual weights, which is highly precise but makes acceleration more difficult.
- **Structured Pruning**: Removes bigger chunks at a time which makes it less accurate compared to unstructured pruning but allows for efficient sparse matrix computation.

Dating back to the 1990s with methods like Optimal Brain Damage(OBD), Modern Pruning is performed via One-shot Pruning or Iterative Pruning to avoid layer collapse [4].

1.2 The Problem

While Pruning is effective, it introduces a potential security vulnerability when coupled with Data Poisoning Attacks. In a targeted data poisoning attack, an adversary modifies a small fraction of the training data points within a bounded limit. The core problem addressed in this work is the vulnerability of pruned networks to optimization-based backdoor attacks that exploit the structural changes caused by aggressive compression.

In a standard *"One-Shot Pruning without fine-tuning"* scenario, a model is trained to convergence and then a significant portion of its weights (e.g., 60%) are permanently removed. We identify that this sudden shift creates a security gap. An attacker can *"trigger"* poisons that remain dormant in the dense, unpruned model but become active immediately after one-shot pruning process. This attack is challenging to detect as the poison appear benign to the full model, causing defense methods that assess model before compression to be ineffective.

Critically, because the model is not fine-tuned after pruning in this pipeline, it lacks the opportunity to recover or adjust its decision boundaries to the loss of connections.

The U-Net, as introduced by Ronneberger et al. [2], is a Convolutional Neural Network mainly used for biomedical image segmentation. The U-shaped architecture consists of a symmetric downsampling path(encoder) and an upsampling path(decoder) connected by skip connections. In this work, we repurposed the U-Net as a U-Net Poison Generator by optimizing poisons that rely on the specific weights destined to be pruned. Once these connections are severed, the subtle adversarial perturbations dominate the remaining neural network paths. This forces the pruned model to collapse into the targeted misclassification state, while the unpruned victim model suppresses the trigger and maintains the expected accuracy.

1.3 The Solution

Existing data poisoning and backdoor attacks assume that pruning can mitigate the adversarial behavior by removing the malicious connections. However, we present that pruning can instead activate and trigger poisons that were benign to the full model. Prior work has not studied backdoor attacks that activates when the model is pruned.

In this paper, we propose a novel Data Poisoning Attack on Pruned Neural Networks. We attempt to create specific malicious training samples (poisons) that are benign to a full-capacity model but cause targeted misclassifications in a pruned model.

Our approach involves using a U-Net model as a poison generator. The U-Net model is trained with two objectives. The first is to minimize the loss for the unpruned model with the poisoned data and clean labels. The second objective is to minimize the loss for the pruned model with the poisoned data and adversarial labels for a specific target set. Data points that do not belong to the target set will retain their clean labels.

With this general framework we introduce two different poisoning strategies. This first strategy is called Approach 1 and the second is Approach 2.
In Approach 1 we specify our target set and our poison set. We then experiment using different datasets and poison generator models. In Approach 2, poison data is not selected before training as in approach 1 rather the data points that are ambiguously labeled from surrogate model are selected inside the alternate training loop. There is no explicit target set in Approach 2.

Our analysis focuses on one-shot pruning without fine-tuning. Future work should investigate the effectiveness of our attack against fine-tuned pruned models and different pruning approaches.

2 Related Work

2.1 Data Poisoning Attack on Quantized Models

Similar work has been done on exploring data poisoning attacks with other model compression techniques, such as model quantization by Huynh et al [1]. Huynh et al. explores a method to create a dirty label data poisoning attack that, while undetectable when the model is not quantized, is effective once the model is quantized [1]. Huynh et al. discusses the notion of a surrogate classifier model f_θ, which represents the victim's model and has parameters θ, and a trigger generator function B_ψ, which gives the result of adding poison to the original dataset and contains a trainable model with parameters ψ [1]. Huynh et al. defines f_θ using ResNet-18 [1]. B_ψ is defined as the following [1]:

$$B_\psi(x) = x + \eta b_\psi(x) \tag{1}$$

In equation (1), x denotes an image from the original clean dataset, η denotes a multiplier to control how much noise is added to the image, and $b_\psi(x)$ is a U-Net model with parameters ψ that generates noise given a clean image [1].

Huynh et al. formally defines the problem as the following, where y are the correct labels corresponding to the images and y_t are the desired adversarial labels [1]:

$$f_\theta(x) = y \tag{2}$$

$$f_{\theta_{quan}}(x) = y \tag{3}$$

$$f_\theta(B_\psi(x)) = y \tag{4}$$

$$f_{\theta_{quan}}(B_\psi(x)) = y_t \tag{5}$$

As can be observed from equations (2) and (3), the surrogate model, when quantized, should output the same classification labels as when it was not quantized when given a clean dataset [1]. This behavior changes once poison is added to the dataset [1]. When the poisoned dataset is passed into the surrogate model, as shown in equation (4), it outputs correct classification labels like before [1]. However, once the surrogate model is quantized, it will misclassify the data to a different desired label, as shown in equation (5) [1].

In order to craft the desired behaviors for both the surrogate model f_θ and the poison generator model B_ψ, Huynh et al. introduces the concept of alternating training, in which each iteration of the training loop involves selecting a mini-batch from the dataset S, choosing a small subset of that mini-batch to poison (denoted as P), and computing two loss functions to optimize parameters θ and ψ simultaneously [1]. Both loss functions used by Huynh et al. are shown below, with L_{CE} representing the cross entropy loss function [1]:

$$\min_{\theta} \sum_{(x,y)\in\mathcal{S}\setminus\mathcal{P}} \mathcal{L}_{\text{CE}}(f_\theta(x), y) + \sum_{(x,y)\in\mathcal{P}} \mathcal{L}_{\text{CE}}(f_\theta(\mathcal{B}_\psi(x)), y_t) \tag{6}$$

$$\min_{\psi} \sum_{(x,y)\in\mathcal{S}} \left[\mathcal{L}_{\text{CE}}(f_\theta(\mathcal{B}_\psi(x)), y) + \mathcal{L}_{\text{CE}}\big(f_{\theta_{\text{quan}}}(\mathcal{B}_\psi(x)), y_t\big) + \text{regularizer}(\eta\, b_\psi(x))\right] \tag{7}$$

Equation (6) seeks to optimize parameters θ in the surrogate model by computing the cross entropy losses of mapping the output of f_θ on the clean data to the correct labels and mapping the output of f_θ on the poisoned data to the newly assigned labels [1]. Equation (7) seeks to optimize parameters ψ in the poison generator model by computing the cross entropy losses of mapping the output of f_θ on the poisoned data to the correct labels and mapping the output of $f_{\theta_{\text{quan}}}$ on the poisoned data to the newly assigned labels [1].

After training both models simultaneously, the final poisoned dataset, which will be handed to the victim to train their classifier model on, is crafted by extracting a subset P from the dataset S, passing each image in P through the poison generator B_ψ and reassigning it a new classification label y_t, and adding these newly created poisoned data points to the dataset S [1].

This work raises the possibility that a similar technique can be deployed for another model compression technique, in this case model pruning.

2.2 U-Net

We also referenced Ronneberger et al.'s work in defining the structure of U-Net models [2]. While traditionally used for image segmentation, we construct a U-Net to add noise to dataset images as described by Huynh et al. [2, 1].

3 Methodology

To understand the vulnerability of Pruned Neural Networks, we developed two unique adversarial attacks, both approaches adopting a clean label data poisoning technique. The attacker's objective is to use the generated poisons to misclassify only when the model is pruned, thereby activating the backdoor attack, i.e,

- Unpruned Model: Predicts the labels correctly.

$$f_\theta(B_\psi(x)) = y \tag{8}$$

- Pruned Model: Misclassify the labels after applying One-shot Pruning.

$$f_{\theta_{pruned}}(B_\psi(x)) = y_t \tag{9}$$

For both approaches, we adapted the methodology of Huynh et al. to employ alternating training to optimize a surrogate model and a poison generator model, replacing model quantization with model pruning and modifying both models and the loss functions to meet our needs for clean label data poisoning (as opposed to dirty label data poisoning).

3.1 Approach 1

In approach 1, we use the notions of a target class and a poison class [3]. The goal is to add poison to the images in the poison class such that they influence images in the target class to be misclassified to match the class label of the poison class [3]. Such misclassification should only happen when the surrogate model is pruned. In our experiments, we identified a subset of same-class images from the training dataset S as the poison class and a subset of same-class images, which are different from the poison class and are separate from the training dataset S, as the target class [3]. We refer to both of these subsets as S_{poison} and S_{target} respectively. We formalize this relationship via the following equations:

$$B_\psi(x) = \begin{cases} x & \text{when } x \notin S_{\text{poison}} \\ x + \eta b_\psi(x) & \text{when } x \in S_{\text{poison}} \end{cases} \tag{10}$$

$$y_t = \begin{cases} y & \text{when } x \notin S_{\text{target}} \\ y_{\text{poison}} & \text{when } x \in S_{\text{target}} \end{cases} \tag{11}$$

We also define two loss functions to minimize that adapt the loss functions proposed by Huynh et al. [1]:

$$\min_\theta \sum_{(x,y)\in S} L(f_\theta(B_\psi(x)),\, y) \tag{12}$$

$$\begin{aligned} \min_\psi \sum_{(x,y)\in S} L_{\text{CE}}(f_\theta(B_\psi(x)),\, y) + \sum_{(x,y)\in S_{\text{target}}} L_{\text{CE}}(f_\theta(B_\psi(x)),\, y) \\ + \sum_{(x,y)\in S} L_{\text{CE}}\big(f_{\theta_{\text{pruned}}}(B_\psi(x)),\, y\big) + \sum_{(x,y)\in S_{\text{target}}} L_{\text{CE}}\big(f_{\theta_{\text{pruned}}}(B_\psi(x)),\, y_{\text{poison}}\big) \end{aligned} \tag{13}$$

Equation (12) minimizes the loss of the surrogate model by calculating the cross entropy loss of mapping the output of f_θ on the dataset S (which includes the subset S_{poison}) to the correct class labels. This mimics how the victim would be training their model on the poisoned dataset. Equation (13) minimizes the loss of the poison generator model by calculating the cross entropy losses of both the unpruned and pruned surrogate models, in which only the images in the subset S_{target} should map to the same label as the poison class when the model is pruned. For simplicity, we used random unstructured pruning to prune 30% of parameters; a seed is defined so that the same parameters are pruned from the surrogate model during each iteration of the training loop.

For our surrogate model, we chose to use a simpler architecture consisting of convolutional layers, a max pool layer, and fully connected linear layers. For the model used in the poison generator function, we tried two different architectures: a traditional U-Net construction and an "oversimplified" version made of two convolutional layers and a ReLU layer in between.

Now that the key differences between Huynh et al.'s setup [1] and our setup have been explained, we can fully outline the steps of our approach. After selecting the images to be in our poison class and target class, we perform alternating training, in which each iteration of the training loop calculates the loss functions given by equations (12) and (13). Afterwards, we used the trained poison generator to poison the clean data (specifically images belonging to the poison class). We then trained a surrogate model from scratch on the poisoned data, using a surrogate model trained on the clean data as our baseline for comparing results.

3.2 ALPS (Ambiguous Label Poisoning Strategy) - Approach 2

Our method involves producing a target poisoning set by identifying samples through label ambiguity and perturbing them using the noise generator model. This consists of 5 major steps: (1) Training the surrogate model on clean data (2) Poison data selection and alternate training. (3) Producing the poison data using noise generator. (4) Training the victim model and pruning (5) Evaluating and comparing the victim model and the pruned victim model.

Training surrogate model on clean data:
First, the surrogate model is trained using the clean data. This is done to improve selection of poison data, which depends on the surrogate model results.

Poison data selection and alternate training:
Within the training loop, the poison data is selected in such a way that the top two class probabilities are closer. The label of the poison data i.e the top k% data points whose top two class probabilities are closer is changed to the class with second highest probability and for other datapoints the label remains unchanged. This is done for two major reasons:

- Such data is closer to the decision boundary and easier to misclassify by adding small pertubations.
- The small perturbations may be caught by the entire model and missed by pruned model (since it is subtle).

This mix of poisoned data and clean data is used to train surrogate model and noise generator alternatively. To meet our needs, the loss functions are formalized as follows:

$$\min_{\theta} \sum_{(x,y)\in S} L(f_\theta(B_\psi(x)),\, y) \tag{14}$$

$$\begin{aligned}\min_{\psi}[\sum_{(x,y)\in S} L_{\text{CE}}(f_\theta(B_\psi(x)),\, y) \quad &+ \sum_{(x,y)\in S\setminus P} L_{\text{CE}}\big(f_{\theta_{\text{pruned}}}(B_\psi(x)),\, y\big) \\ &+ \sum_{(x,y)\in P} L_{\text{CE}}\big(f_{\theta_{\text{pruned}}}(B_\psi(x)),\, y_{\text{poison}}\big)]\end{aligned} \tag{15}$$

Equation (14) and (15) depicts the loss function for modifying surrogate model and noise generator model weight parameters respectively. Equation (14) aims to improve classification by surrogate model even after adding the noise generator. Equation (15) is incorporated in such a way that the noise generator adds poison such that the unpruned model and clean pruned model prodcues correct y label and pruned poison data gets classified as class with second highest probability.

Producing poison data using noise generator:
After training process is completed, the poison data selection is done on the entire data using surrogate model and noise generator is applied on the poisoned data.

Training and evaluating victim model and pruned victim model:
The poisoned data is then used to train the victim model and then is pruned. These models are then evaluated and compared. It is noticeable that there is a huge drop in accuracy of pruned model from unpruned model.

4 Experiments

4.1 Dataset information:

To perform our experiments, we used MNIST datasets, a standard benchmark for handwritten digit recognition. Each image is 28*28 grayscale image associated with labels 0 to 9. The MNIST dataset is approximately class-balanced dataset with 60,000 data points in trainset and 10,000 datapoints in testset.

4.2 Implementation details

Both our methods have been implemented using Pytorch in GPU enabled google collab notebooks.

Approach 1:

Pre-training phase:
Before we begin training, we first randomly sample 1% of the digit 1 images to form our poison dataset. Similarly, we sample 1% of the digit 7 images from the test set which form our target set.

Training phase:
For the main training phase we implement an alternate training scheme. We conduct two sub-experiments in this phase, one with using a modified U-Net as the poison generator and another

with using a plain, double convolutional layer network. We use a simple CNN model to mimic our victim (surrogate) model in both experiments. Both models are optimized using SGD with a learning rate of 0.001 with mini-batch SGD. In alternate training, we first compute the poisons, then evaluate the loss of the victim model with the poisoned mini-batch and update the weights of the victim model. Within the same iteration, we freeze the weights of the victim model and compute the loss for our poison generator and update the weights for the poison generator model.

Post-training phase:
To evaluate the effectiveness of our poisons we begin with training a victim model on clean data and comparing the per-digit accuracy for the pruned and un-pruned model. We repeat the same experiment after training the model on the poison dataset. We should also note that ideally to understand the effectiveness of our poisons we would need to compute the accuracy of our target class that we selected pre-training, however, this would not be any different than checking the accuracy of digit 7 images given the nature of the MNIST dataset.

ALPS (Ambiguous Label Poisoning Strategy) - Approach 2:

Pre-training phase:
The preprocessing involves normalizing data with mean 0 and std 1.

Training phase:
In this method, the data is poisoned at 7% for every alternate training loop. The surrogate model is a simple CNN model, optimized using SGD with learning rate 0.01. The noise generator model simple unet architecture with 1 downsampling and 1 upsampling layer, trained using SGD with learning rate 0.001. Both the models use cross entropy function for loss calculation. It is important to note that the victim model and the surrogate model has same structure, optimizer and loss function, ensuring consistent learning dynamics.

Post training phase:
After we train the UNet Generator for 5 epochs, we construct the `S_pois` consisting of 93% clean data and 7% posioned data. Then, the victim model is trained for 10 epochs on this `S_pois` dataset. After victim model is trained, it is pruned using L1 unstructured with 60% of the weights removed and no fine-tuning performed post pruning. The pruned and unpruned models trained using clean and poison data are evaluated to check if the poison as kicked in as expected.

4.3 Results

Approach 1:

For Approach 1, we first created our baseline with a modified U-Net model as the poison generator and simple CNN network as the victim model. We obtain the per-digit accuracies in Figure 2 for the pruned v/s the unpruned model.

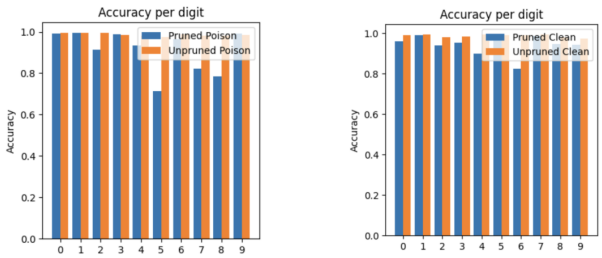

Figure 1: Accuracies for (a) model trained with poisoned data and (b) model trained with clean data

Figure 1(a) on the left has been trained with poison data. After training we remove 30% of the weights and compute the accuracies again to observe if our poison has impacted our model. In the pruned model, our target class is digit 7 images. While there is a drop in accuracy by 20% for the

pruned model with classifying 7, we cannot conclusively say that this drop is produced by our poison. One reason for this is that other digits like 5 also experience a drop in accuracy with the poison and it is possible that the reduction in accuracy is due to model pruning. In future work, we plan to investigate whether the observed drops in accuracy can be attributed to the poison.

We also conducted a sub-experiment which involved replacing the poison generator with a simple 2-convolution layer model. The results for training the victim model with the poison generator are presented in Figure 2.

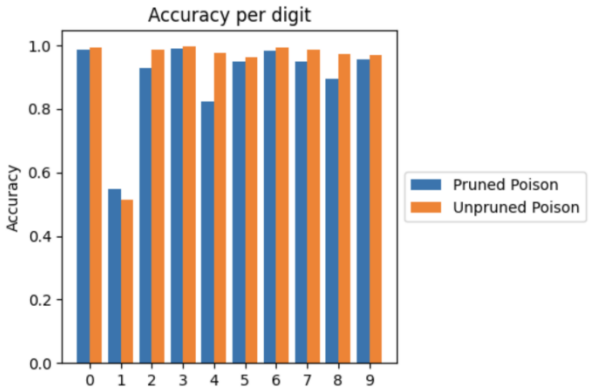

Figure 2: Accuracies for model trained with simple poison model instead of U-Net. This experiment has been labeled as "Oversimplified U-Net" in the project GitHub

Simplifying the poison production model has reduced the accuracy of our poison class, the digit 1 images. Ideally we should have observed a drop in accuracy for the digit 7 images, however, the poison produced by the new poison generator has impacted the poison class itself.

The final sub-experiment we conducted with Approach 1 involved expanding our methodology to the CIFAR-10 dataset post-final presentation. For this part we set our target class to 'dog' images and poison class to 'cat' images. The poison generator is switched back to the modified U-Net in the first sub-experiment. In Figure 3, we have presented a poison image from the 'cat' class in 3(a) and the clean version of that image in 3(b).

Figure 3: (a) shows a poisoned image and (b) on the right shows the clean version of the same image.

The accuracy results for CIFAR-10 are presented in Figure 4.

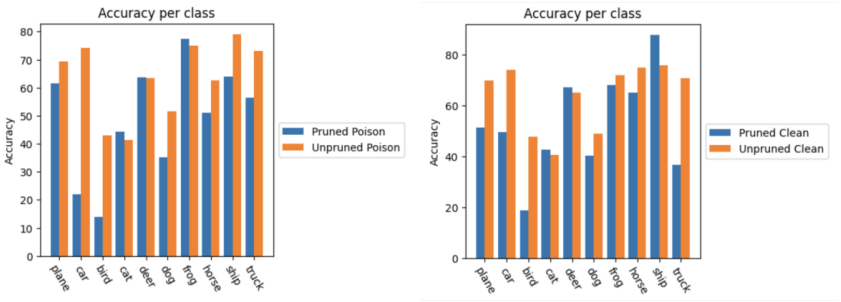

Figure 4: (a) shows results of a model trained on the poisoned data and (b) on the right is for a model trained on clean data

The first point to note here is that CIFAR-10 classification is heavily impacted by pruning. In the clean version in Figure 4(b) we notice significant pruned model accuracy drops compared to the MNIST clean version in Figure 1(b). Our target class, which is the 'dog' class, observes a small reduction in accuracy when pruned. However, we face the issue as before, which is that we cannot conclusively state that the accuracy drops are because of the poison.

ALPS (Ambiguous Label Poisoning Strategy) - Approach 2:

Similar to training, the pruning rate used for evaluation is also 60%, done using L1 unstructured pruning. Unpruned clean data model refers to surrogate model that is trained on clean data. Pruned clean data model refers to pruned version of surrogate model on clean data. Unpruned poison data model refers to surrogate model that is trained on poisoned data. Pruned poison data model refers to pruned version of surrogate model on poisoned data.Table 1 depicts the accuracy for unpruned clean data model, pruned clean data model, unpruned poisoned data model and pruned poisoned data model.

Model State	**Accuracy (%) after 5 epochs**	**Accuracy (%) after 10 epochs**
Unpruned clean data model	97.66	97.75
Pruned clean data model	96.10	97.01
Unpruned poisoned data model	97.63	97.61
Pruned poisoned data model	91.93	85.56

Table 1: Performance of the victim model before and after pruning.

Figure 6 and Figure 7 depicts the accuracy comparison and confusion matrix of unpruned and pruned data trained on clean and poison data after training for 5 epochs. Figure 8 and Figure 9 depicts the accuracy comparison and confusion matrix of unpruned and pruned data trained on clean and poison data after training for 10 epochs.

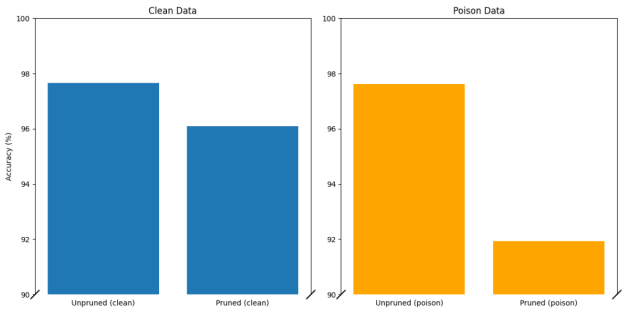

Figure 5: Accuracy comparison of pruned and unpruned model on clean and poison data after training for 5 epochs

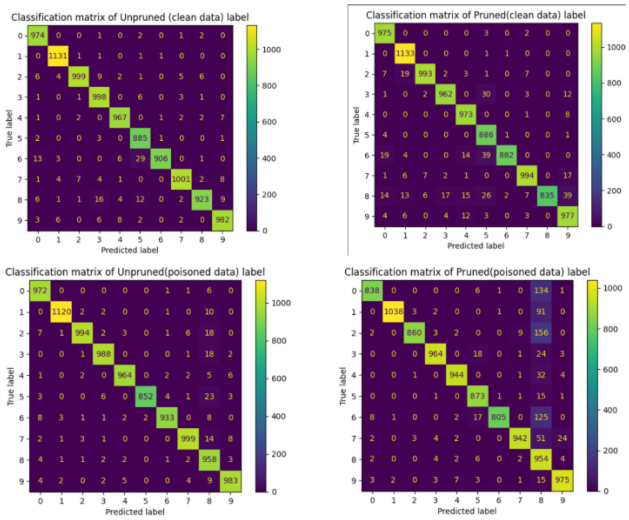

Figure 6: Confusion matrix of pruned and unpruned model on clean and poison data after training for 5 epochs

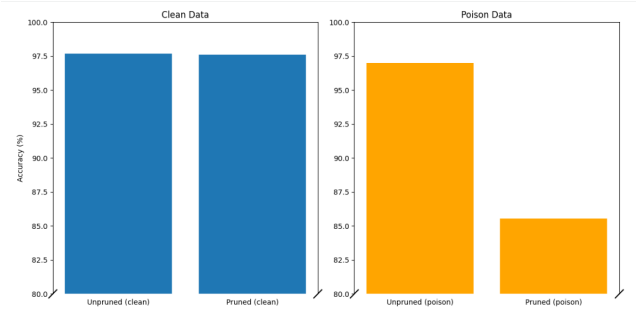

Figure 7: Accuracy comparison of pruned and unpruned model on clean and poison data after training for 10 epochs

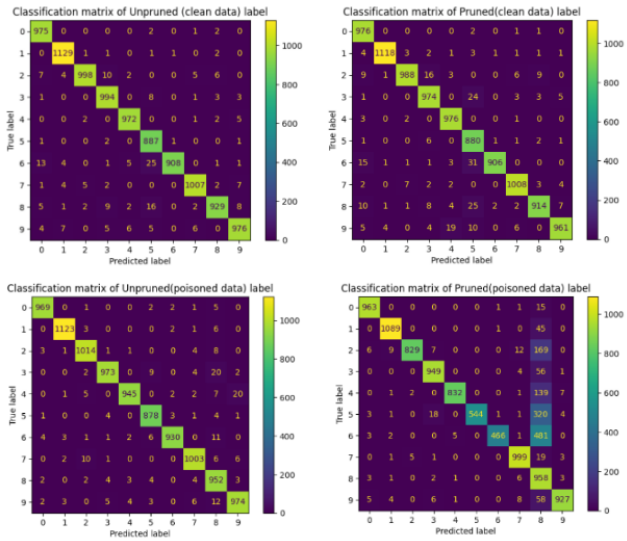

Figure 8: Confusion matrix of pruned and unpruned model on clean and poison data after training for 10 epochs

The comparable accuracy drop in clean data case and huge drop in accuracy for poisoned data suggests that the poisoned data may have only affected the pruned model and not the unpruned model that is trained on poison data.

5 Conclusion

In this work, we demonstrated that pruned neural networks are vulnerable to security threats compared to the full model. By choosing specific poisons, we can embed a backdoor that is benign to the full model but gets activated when the model is pruned. Our attack through two unique approaches demonstrates a clear security breach where the victim model maintains a high accuracy in its dense state masking the poison effectively but after applying L1 unstructured pruning, the model's accuracy goes down significantly. This discrepancy shows that clean label attacks can remain benign until the model is compressed, allowing attackers to take advantage of this vulnerability and trigger poisons to attack the victim's model.

While our work signifies the need for security auditing for compressed models, our current attack makes several assumptions. When constructing our poison, we assumed that the attacker is aware of the victim's model architecture, the type of pruning used by the victim, and that the victim will not fine-tune the model after one-shot pruning. In future work, we must explore the impacts of fine-tuning which may nullify the effects of the poison. Additionally, to generalize our attack we can also consider different methods for pruning and poisoning. One idea we formulated involves using Witches' Brew gradient matching to generate a poison for the lottery-ticket of a randomly initialized model [4, 5]. If the victim and the attacker have the same initialized network, chances are that they may discover similar lottery-ticket sub-networks, i.e. sub-networks that share a certain number of parameters. Since gradient matching poisoning is parameter and gradient specific, we could design poisons specifically for the pruned, lottery-ticket model.

Despite the limitations in Approach 1 and Approach 2, our project has demonstrated there is a vast array of potential security vulnerabilities that can stem from model pruning and that there is a need for security verification from initial training to post-training covering the entire deployment pipeline.

6 Contributions

Ideation:

All members (Diya, Anvi, Janani and Sherine) contributed to formulating and choosing ideas for the project, including finding relevant papers and attending team meetings.

Experiments:

Diya and Anvi collaborated in writing all code corresponding to Approach 1. Janani and Sherine collaborated in writing all code corresponding to Approach 2. All members assisted with debugging each others' code when needed. It is also worth noting that relevant works had no GitHub source code, meaning we had to write all code based on any pseudocode and explanations provided in these papers. Additionally, we want to acknowledge the use of AI assistance, such as ChatGPT, in writing small code snippets and for debugging; all notebooks in our GitHub repository indicate via comments which code blocks were written with AI assistance or reference to another source.

Writeup:

The following list outlines which sections were written by which members:

- Abstract written by Diya
- Introduction written by Sherine and Diya
- Related Work written by Anvi
- Methodology written by Sherine, Anvi, and Janani
- Experiments written by Diya and Janani
- Conclusion written by Sherine and Diya

All members helped with proof reading others' sections as needed.

References

[1] Huynh, T., Tran, A., Doan, K.D., Pham, T. (2025). Data Poisoning Quantization Backdoor Attack. In: Leonardis, A., Ricci, E., Roth, S., Russakovsky, O., Sattler, T., Varol, G. (eds) Computer Vision – ECCV 2024. ECCV 2024. Lecture Notes in Computer Science, vol 15142. Springer, Cham. https://doi.org/10.1007/978-3-031-72907-2_3

[2] Ronneberger, O., Fischer, P., & Brox, T. (2015). U-Net: Convolutional networks for biomedical image segmentation. arXiv. https://arxiv.org/abs/1505.04597

[3] Mirzasoleiman, B. (2025). 260D_Lecture_10 [PDF Slides]. Bruin Learn. https://bruinlearn.ucla.edu/

[4] Mirzasoleiman, B. (2025). 260D_Lecture_13 [PDF Slides]. Bruin Learn. https://bruinlearn.ucla.edu/

[5] Geiping J., Fowl L., W. Ronny Huang, Czaja W., Taylor G., Moeller M., Goldstein T. (2021). Witches' Brew: Industrial Scale Data Poisoning via Gradient Matching. arXiv. https://arxiv.org/abs/2009.02276